AI Engineering and Claude Architect · Part V, Certification

Certification, Lesson 5

Final Revision Handbook

All 30 task statements in one read. This holds the decision each one tests, its traps, and every known difference between the guide's answer and today's products.

Module	AI13 of 13
Lesson	5 of 5
Prerequisites	Everything in the required path
Exam relevance	**All 30 task statements**
Reading time	~45 minutes. Read it twice in the final week

Contents

1. Final Revision Handbook

Lesson 5 of 5 · Certification

Final Revision Handbook

Key idea

Guarantees go in code. Judgment stays with the model. Context must be passed, trimmed and preserved on purpose. Measure by segment, not on average. Nearly every item in the exam guide comes back to one of those four ideas.

What this is. A condensed form of the guide's objectives and this course's lessons. It teaches nothing new, so if a row is unfamiliar, go to the lesson named beside its section. The "Trap" columns are this course's analysis, not guide text.

Domain 1: Agentic Architecture and Orchestration (27%)

TS	Decision	Answer	Trap
1.1 Loops	When to continue	`stop_reason`: `tool_use` continues, `end_turn` ends	Parsing text, iteration caps as the main stop, "has text" checks
1.1	Tool results	Append to history for the next request	Fresh conversation per step
1.1	Who decides the next tool	The model, from context	Pre-set sequences for open-ended work
1.2 Coordinator	Topology	Hub and spoke: all traffic via coordinator	Direct subagent-to-subagent
1.2	Which subagents	Choose by query complexity	Always the full pipeline
1.2	Missing subtopics	Decomposition was too narrow	Blaming the search subagent
1.2	Scope split	Distinct subtopics or source types	Overlap
1.2	Quality	Evaluate synthesis for gaps, re-delegate, re-synthesize	One pass only
1.3 Spawning	How	`allowedTools` includes `"Task"`	Assuming subagents just run
1.3	Context	Pass complete findings in the prompt	Assumed inheritance or shared memory
1.3	Attribution	Structured: content separate from metadata	Merged prose
1.3	Parallel	Multiple Task calls in **one** response	Across separate turns
1.3	Instructions	Goals and quality criteria	Step-by-step procedures
1.3	Definition	`AgentDefinition`: description, prompt, tools	
1.4 Workflows	Ordering	Programmatic prerequisite	"Always verify first" in a prompt

TS	Decision	Answer	Trap
1.4	Multi-concern	Split, investigate in parallel with shared context, unify	First issue only
1.4	Handoff	Customer ID, root cause, amount, recommended action	Assuming a human can read the transcript
1.5 Hooks	Normalize formats	PostToolUse	Prompt instructions
1.5	Block by policy	Call-interception hook, then redirect to escalation	Trust the model
1.5	Hook or prompt	Guaranteed means hook	Prompt for compliance
1.6 Decomposition	Predictable review	Prompt chaining	Dynamic plan
1.6	Open-ended investigation	Adaptive subtasks	Fixed sequence
1.6	Large review	Per-file passes + cross-file pass	One giant prompt
1.6	Legacy test coverage	Map, find high impact, prioritize, adapt	Start at file 1
1.7 Sessions	Continue named work	`--resume <session-name>`	Re-explore
1.7	Compare approaches	`fork_session` from a shared baseline	Two re-analyses
1.7	Code changed since	Tell it which files changed	Assume it knows
1.7	Results stale	New session + structured summary	Resume anyway

Lessons: AI06 L1, L2, L6, L7, L8, and AI13 L2.

Domain 2: Tool Design and MCP Integration (18%)

TS	Decision	Answer	Trap
2.1 Tools	Misrouting	Better descriptions: inputs, examples, edges, when not to use	Classifier or few-shot first
2.1	Overlap	Rename and redefine (`extract_web_results`)	Merge into a vaguer tool
2.1	Generic tool	Split with clear contracts	One tool with mode flags
2.1	Still misrouting	Check system prompt for keyword associations	Only editing tools
2.2 Errors	Failure signal	`isError`, and structured metadata	"Operation failed"
2.2	Categories	Transient, validation, business, permission	One error type
2.2	Retry	`isRetryable`, and false for policy violations	Retry everything
2.2	Business rule	`retriable: false` and a customer-friendly explanation	Raw errors
2.2	Where to recover	Locally for transient errors, then propagate with partial results	Every error to the coordinator
2.2	No matches	Successful and empty	Treating it as an error, or an outage as empty
2.3 Distribution	Tool count	Few per agent (4-5, against 18)	All tools everywhere
2.3	Misuse	Scope to the role	Prompt not to
2.3	Occasional cross-role	Scoped `verify_fact`	Full search for synthesis
2.3	Constraint	`load_document` over `fetch_url`	Generic fetch

TS	Decision	Answer	Trap
2.3	`tool_choice`	`auto`: maybe text. `any`: some tool. Named: that tool	Confusing them
2.3	Do this first	Force the tool, then later turns	Asking nicely
2.4 MCP	Team servers	`.mcp.json`, with `${VAR}`	Committed secrets
2.4	Personal servers	`~/.claude.json`	Project file
2.4	Discovery	All tools found at connection, available together	Per-request loading
2.4	Catalogs	MCP resources	Exploratory tool calls
2.4	Prefers Grep	Describe the MCP tool better	Remove Grep
2.4	Standard integration	Community server first	Custom build
2.5 Built-ins	Search contents	Grep	Glob
2.5	Find files by name	Glob	Grep
2.5	Targeted change	Edit, with unique text	
2.5	Edit fails, not unique	Read + Write	Retrying Edit
2.5	Learn code	Grep, then Read and follow imports	Read everything
2.5	Wrappers	Find exported names, then search each	One name only

Lessons: AI06 L3, L4, L5, AI12 L1, and AI13 L3.

Domain 3: Claude Code Configuration and Workflows (20%)

TS	Decision	Answer	Trap
3.1 Memory	Levels	User, project, directory	
3.1	Teammate doesn't get it	It's in `~/.claude/`, so move it to project	Tell them to copy it
3.1	Modular	`@import` for each package's standards	Paste it all
3.1	Large file	Split into `.claude/rules/` topic files	Larger `CLAUDE.md`
3.1	What loaded	`/memory`	Guessing
3.2 Skills	Commands	`.claude/commands/` shared, `~/.claude/commands/` personal	Wrong scope
3.2	Skills	`.claude/skills/<name>/SKILL.md`	
3.2	Verbose output	`context: fork`	Compacting later
3.2	Tool limits	`allowed-tools`	
3.2	Needs a parameter	`argument-hint`	
3.2	Personal variant	`~/.claude/skills/` under a **different name**	Editing the shared skill
3.2	Skill or memory	On demand vs always loaded	Everything always loaded
3.3 Rules	Conditional	YAML `paths` with globs	
3.3	Loading	Only when editing matching files	Always loaded
3.3	Spread files	Glob rule, like `**/*.test.tsx`	Directory `CLAUDE.md`
3.4 Plan	Plan when	Large changes, many valid approaches, architecture, multi-file	Plan for every change

TS	Decision	Answer	Trap
3.4	Direct when	Clear, small scope	Plan for a one-line fix
3.4	Verbose discovery	Explore subagent	Filling main context
3.4	Combining	Plan, then execute directly	Either alone
3.5 Iteration	Inconsistent results	2-3 input/output examples	More prose
3.5	Test-first	Write tests, share failures	Vague feedback
3.5	Unfamiliar domain	Interview pattern	Jumping in
3.5	Edge case	Specific case with input and expected output	"Handle nulls better"
3.5	Feedback order	Interacting: together. Independent: one by one	Always one way
3.6 CI	Non-interactive	`-p`	Waiting for input
3.6	Parse	`--output-format json` + `--json-schema`	Regex
3.6	Context	`CLAUDE.md`	Pasted every time
3.6	Self-review	Separate instance	Same session
3.6	Re-review	Include prior findings, report new only	Duplicates
3.6	Test gen	Give existing tests and fixture docs	Duplicates, low value

Lessons: AI12 L1–L5, and AI13 L2.

Domain 4: Prompt Engineering and Structured Output (20%)

TS	Decision	Answer	Trap
4.1 Criteria	Precision	Categorical criteria: report X, skip Y	"Be conservative", "high confidence only"
4.1	Trust	Disable a noisy category while fixing it	Accept the noise
4.1	Severity	Criteria with code examples per level	Adjectives
4.2 Few-shot	Consistency	2-4 targeted examples	Long instructions
4.2	Ambiguity	Show reasoning: why this, not that	Bare input and output
4.2	False positives	Acceptable pattern vs real issue	Bad examples only
4.2	Extraction	Varied layouts, informal values, absent fields	Single format
4.3 Schemas	Reliability	Tool use with a schema	"Respond in JSON"
4.3	Tool choice	Unknown type: `any`. A step first: named	`auto` when a call is required
4.3	Limits	Stops syntax errors, not semantic ones	Assuming it validates meaning
4.3	Absent data	Nullable	Required, leading to invention
4.3	Categories	"other" + detail, "unclear"	Exhaustive enums
4.3	Formats	Normalization rules in the prompt	Schema alone
4.4 Validation	Retry	Document + failed output + specific errors	"Try again"
4.4	Limits	Fails if data is absent or external	More retries
4.4	Feedback	`detected_pattern`, for dismissal analysis	Free text

TS	Decision	Answer	Trap
4.4	Self-check	`calculated_total` + `stated_total`, `conflict_detected`	One total
4.5 Batches	Facts	50% off, up to 24 h, no SLA, no multi-turn tools	Assuming lower latency
4.5	Fit	Overnight or weekly, not pre-merge	Batch for blocking
4.5	Failures	Only failed `custom_id`s, modified	Redo all
4.5	Before scale	Refine on a sample	Iterating at full volume
4.5	SLA	Window + 24 h ≤ SLA	Ignoring worst case
4.6 Review	Self-review	Fresh independent instance	"Check your work", more thinking
4.6	Large PRs	Per-file + integration passes	One pass
4.6	Routing	Self-reported confidence per finding, calibrated	Filtering by it, uncalibrated

Lessons: AI03 L2, L3, L5, AI12 L5, and AI13 L3.

Domain 5: Context Management and Reliability (15%)

TS	Decision	Answer	Trap
5.1 Context	Numbers survive	Case facts outside the summary	Progressive summarizing
5.1	Long inputs	Key summary first, headers throughout	Important items in the middle
5.1	Verbose tools	Trim to relevant fields	Keep the 40 fields
5.1	Multi-issue	A separate structured layer	One narrative
5.1	Limited budget	Upstream returns structured data	Verbose chains
5.1	History	Pass the full history each request	Assume the server remembers
5.2 Escalation	Triggers	Human requested, policy gap, no progress	Sentiment, self-reported confidence
5.2	"I want a person"	Immediately	Investigating first
5.2	Frustrated, simple	Acknowledge and resolve, escalate if they repeat	Instant escalation
5.2	Silent policy	Escalate	Improvising
5.2	Multiple matches	Ask for identifiers	Guess
5.2	Calibration	Criteria with few-shot examples	Vague instruction
5.3 Errors	Coordinator needs	Type, attempted query, partial results, alternatives	"Search unavailable"
5.3	Empty vs failed	Distinguish them	Same result for both
5.3	Anti-patterns	Silent suppression, and killing the workflow	Either extreme
5.3	Reports	Coverage annotations	Silent gaps
5.4 Exploration	Symptom	"Typical patterns", inconsistent answers	Trusting it
5.4	Persist	Scratchpad files	Memory alone

TS	Decision	Answer	Trap
5.4	Isolate	Subagents for narrow questions	Main session reads all
5.4	Phases	Summarize, inject into next phase	Lost findings
5.4	Crash recovery	Exported state + manifest on resume	Restart
5.4	Full session	`/compact`	Continuing to degrade
5.5 Review	Metric	By document type and field	97% overall
5.5	Ongoing	Stratified random sample of high-confidence output	No sampling
5.5	Thresholds	Field-level, calibrated on labeled data	Raw score
5.5	Limited reviewers	Low-confidence or conflicting docs first	Random review
5.6 Provenance	Keep sources	Claim-source mappings through synthesis	Plain summary
5.6	Conflicts	Annotate both, with sources	Pick one
5.6	Dates	Require them	Reading a time gap as a conflict
5.6	Report	Established vs contested, by content type	One uniform format

Lessons: AI03 L6, AI05 L4, AI06 L5–L8, AI10 L3, and AI12 L4.

Guide answers vs what to build today

Answer the exam from the left column. Everything here was checked against current docs, on the date in `references.md`. **This is the complete list this course found, and it may not be everything.**

Topic	Exam's answer	Build today
Spawning subagents	Task tool, and `"Task"` in `allowedTools`	Agent tool. `system:init` still lists Task
Commands and skills	Separate	Merged, and both create `/x`
`allowed-tools`	Restricts during a skill	Pre-approves for a turn. Use deny rules to block
`tool_choice` forced	`any` and named guarantee a call	Returns 400 on some current models. Use `auto` with strict tools and an instruction, or structured outputs
Structured output	Tool use is most reliable	`output_config.format` also exists
Errors	`errorCategory`, `isRetryable`	Only `isError` is protocol. The rest is your own design
`stop_reason`	`tool_use`, `end_turn`	Also handle `max_tokens`, `pause_turn`, `refusal`
Imports	`@import`	`@path`
Grep, Glob	Built-in tools	Absent by default on macOS, Linux and WSL, unless named
Edit fallback	Read + Write	Also a longer anchor, or `replace_all`
CI context	`CLAUDE.md`	True of plain `-p`. `--bare` skips it
Subagent context	Never inherited	A fork subagent inherits it
Temperature	Not in the guide's objectives	Rejected or restricted on current models. Don't choose it as a fix

Numbers and names to know

Item	Value
Batch discount and window	50%, up to 24 hours
Exam	60 items, 120 min, 4 of 6 scenarios, scaled 720
Few-shot	2-4 targeted examples
Refinement examples	2-3 input/output
Tool count example	4-5, not 18
Domain weights	27, 18, 20, 20, 15
Files	`.mcp.json`, `~/.claude.json`, `CLAUDE.md`, `.claude/rules/`, `.claude/commands/`, `.claude/skills/<name>/SKILL.md`
Frontmatter	`paths`, `context: fork`, `allowed-tools`, `argument-hint`
Fields	`custom_id`, `errorCategory`, `isRetryable`, `detected_pattern`, `calculated_total`, `stated_total`, `conflict_detected`
Flags	`-p`, `--output-format json`, `--json-schema`, `--resume`, `fork_session`
Hooks	PostToolUse, plus tool call interception

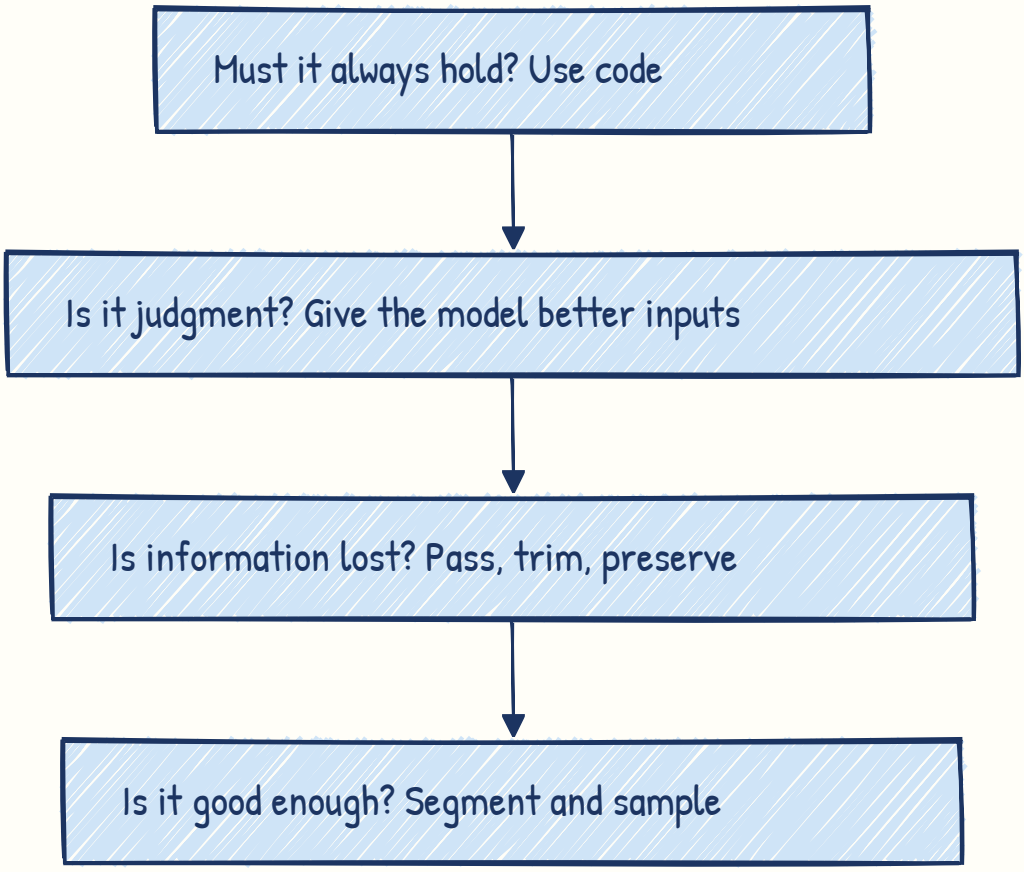


Figure 1. A fast triage for an item you find hard. Ask these four questions in turn: is it a guarantee, a judgment, a context question or a measurement?

Out of scope, per the guide

Don't spend the final week on these. The guide says these related topics won't appear on the exam. They're valuable for your work, and the course teaches many of them:

- → Fine-tuning Claude models or training custom models.
- → Claude API authentication, billing or account management.
- → Detailed implementation of specific languages or frameworks, beyond what's needed for tool and schema configuration.
- → Deploying or hosting MCP servers (infrastructure, networking, container orchestration).
- → Claude's internal architecture, training process or model weights.
- → Constitutional AI, RLHF or safety training methodologies.
- → Embedding models or vector database implementation details.
- → Computer use (browser automation, desktop interaction), and vision or image analysis.

→ Streaming API implementation or server-sent events.

→ Rate limiting, quotas or API pricing calculations.

→ OAuth, API key rotation or authentication protocol details.

→ Specific cloud provider configurations (AWS, GCP, Azure).

→ Performance benchmarking or model comparison metrics.

→ Prompt caching implementation details, beyond knowing it exists.

→ Token counting algorithms or tokenization specifics.

Your error log: the items you keep missing

Revision sheet

→ Guarantees go in code, and judgment stays with the model.

→ Context must be passed, trimmed and preserved on purpose.

→ Measure by segment, not on average.

→ Answer from the guide, and build from current docs.

→ Read the question, mark the constraint, eliminate, pick the root cause.